```
        flag(ATOMIC_FLAG_INIT)
    {}
    void lock()
    {
        while(flag.test_and_set(std::memory_order_acquire));
    }
    void unlock()
    {
        flag.clear(std::memory_order_release);
    }
};
```

Such a mutex is very basic, but it's enough to use with `std::lock_guard<>` (see chapter 3). By its very nature it does a busy-wait in `lock()`, so it's a poor choice if you expect there to be any degree of contention, but it's enough to ensure mutual exclusion. When we look at the memory-ordering semantics, you'll see how this guarantees the necessary enforced ordering that goes with a mutex lock. This example is covered in section 5.3.6.

`std::atomic_flag` is so limited that it can't even be used as a general Boolean flag, because it doesn't have a simple nonmodifying query operation. For that you're better off using `std::atomic<bool>`, so I'll cover that next.

5.2.3. Operations on std::atomic<bool>

The most basic of the atomic integral types is `std::atomic<bool>`. This is a more full-featured Boolean flag than `std::atomic_flag`, as you might expect. Although it's still not copy-constructible or copy-assignable, you can construct it from a nonatomic `bool`, so it can be initially `true` or `false`, and you can also assign to instances of `std::atomic<bool>` from a nonatomic `bool`:

```
std::atomic<bool> b(true);
b=false;
```

One other thing to note about the assignment operator from a nonatomic `bool` is that it differs from the general convention of returning a reference to the object it's assigned to: it returns a `bool` with the value assigned instead. This is another common pattern with the atomic types: the assignment operators they support return values (of the corresponding nonatomic type) rather than references. If a reference to the atomic variable was returned, any code that depended on the result of the assignment would then have to explicitly load the value, potentially getting the result of a modification by another thread. By returning the result of the assignment as a nonatomic value, you can avoid this additional load, and you know that the value obtained is the actual value stored.

Rather than using the restrictive `clear()` function of `std::atomic_flag`, writes (of either `true` or `false`) are done by calling `store()`, although the memory-order semantics can still be specified. Similarly, `test_and_set()` has been replaced with the more general `exchange()` member function that allows you to replace the stored value with a new one of your choosing and atomically retrieve the original value. `std::atomic<bool>` also supports a plain nonmodifying query of the value with an implicit conversion to plain `bool` or with an